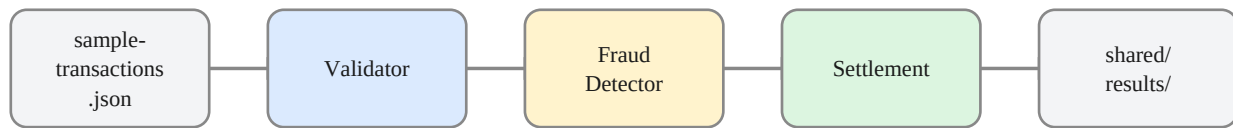


Transaction Processing Pipeline

Homework 6 Capstone — Created by Illia Chantsov

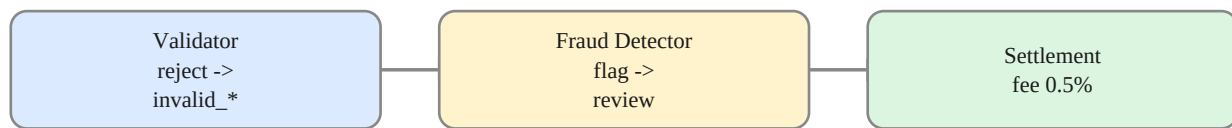
What It Does

- Validates, screens for fraud, and settles a batch of transactions from `sample-transactions.json`.
- Three sequential stages, communicating via a file-based envelope protocol.
- Every transaction ends in exactly one terminal outcome: `settled`, `flagged_for_review`, or `rejected`.
- A read-only web dashboard and a custom MCP server both expose the results.



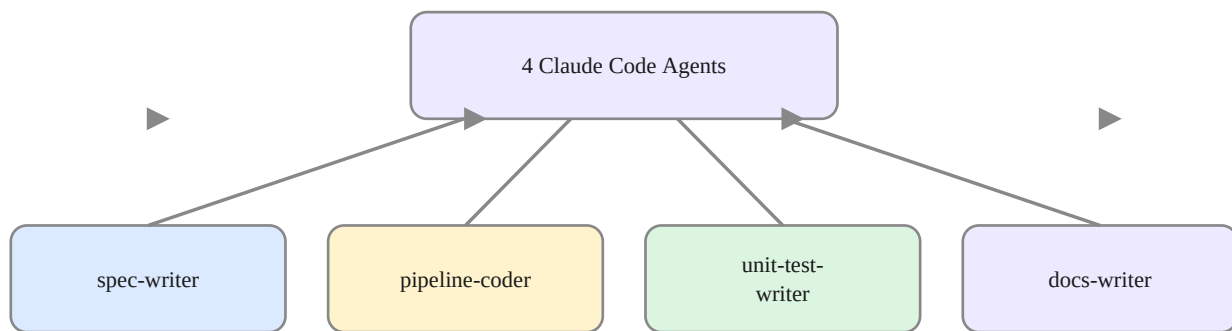
Pipeline Stages

- Validator — required fields, positive Decimal amount, ISO 4217 currency. Rejects with a specific reason.
- Fraud Detector — risk score 0-100 from high value (>\$10,000), cross-border, and unusual-timing signals; score ≥ 50 is flagged for manual review instead of auto-settling.
- Settlement — 0.5% flat fee (ROUND_HALF_UP), settlement batch dated transaction date + 1 day.



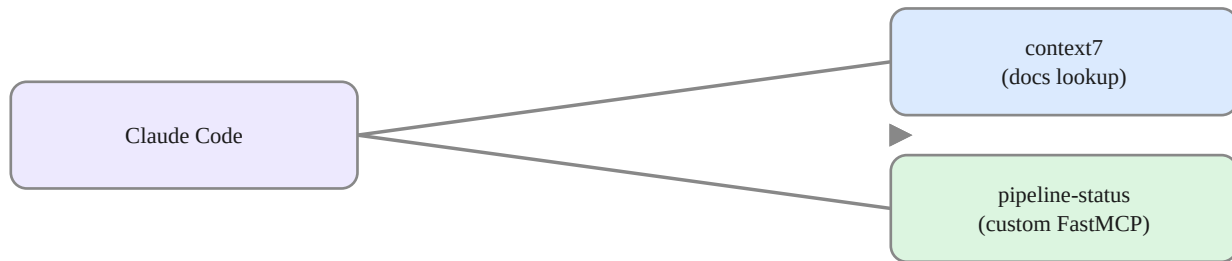
AI Workflow (4 Agents)

- spec-writer -> specification.md + agents.md, via /write-spec.
- pipeline-coder -> pipeline code, using context7 for framework lookups.
- unit-test-writer -> tests/, coverage gated at 80% by a Claude Code PreToolUse hook.
- docs-writer -> README.md and HOWTORUN.md.
- All 4 registered as real Claude Code subagents in .claude/agents/ — not standalone scripts.



MCP Integration

- context7 — general framework/library documentation lookups during code generation.
- pipeline-status (custom FastMCP server): get_transaction_status, list_pipeline_results, resource pipeline://summary.
- Both configured together in .mcp.json.



```
What's the current recommended way to define a resource template in FastMCP?

Called context7

The current recommended way (per FastMCP's docs, fetched just now via context7): use the @mcp.resource decorator with {param} placeholders in the URI string, mapped directly to function arguments - e.g.:

@mcp.resource("weather://{city}/current")
def get_weather(city: str) -> str:
    ...

FastMCP resolves the placeholder from the URI directly into the matching parameter. A few extras worth knowing:
- Wildcards: {param} captures multiple path segments (e.g. {city} matches "London" or "London/England").
- Query parameters: RFC 6579 syntax like {param}/{value} lets you add optional params - they must have default values, e.g. def get_detail(id: str, format: str = "json").
- Static vs. template: a plain fixed URI (no {}) is a static resource; anything with {param} is a template - list_resources distinguishes the two, templates show up with a uri_template field.

In our own mcp/server.py, pipeline://summary is a static resource (no placeholders needed since it always returns "the latest summary"), which is why it's just @mcp.resource("pipeline://summary") with a zero-argument function - no template syntax required there.

Brewed for 55s

What's the current status of transaction TXN002?

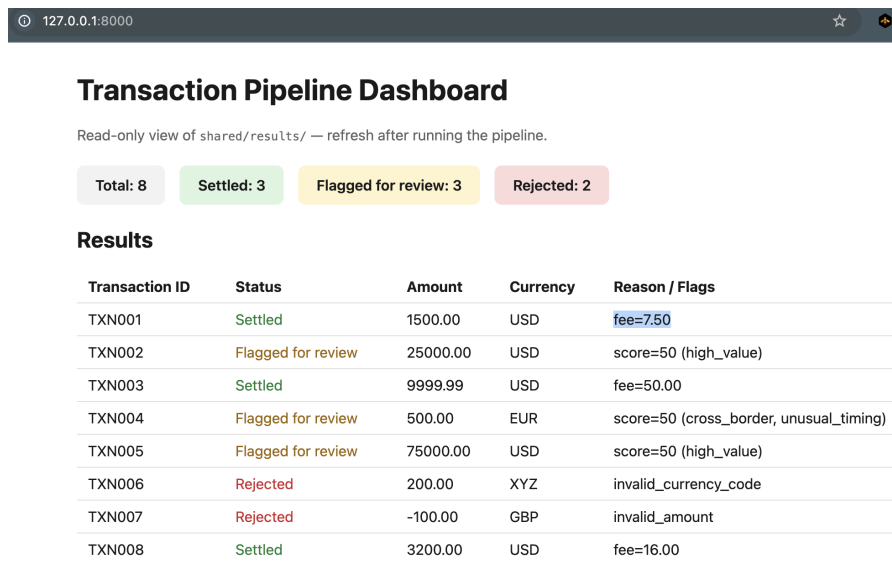
Called pipeline-status

TXN002 is currently flagged_for_review - it hasn't been settled. It's a $25,000.00 USD wire transfer ('Equipment purchase'), and it was flagged with a risk score of 50 due to the high_value signal alone (amount over the $10,000 threshold). Since it's flagged, not cleared, it never reached the settlement stage - no fee or net amount has been computed for it, and it's sitting as a terminal result awaiting manual review.
```

A live context7 lookup and a custom MCP tool call, captured this session.

Demo Walkthrough

- python3 orchestrator.py runs all 8 sample transactions to completion: 3 settled, 3 flagged_for_review, 2 rejected.
- The web dashboard shows the same breakdown live.



The live dashboard after a pipeline run.

Testing & Coverage

- 38 tests across pipeline stages, the orchestrator (integration), the frontend, and the MCP server — isolated from the real shared/ directory via tmp_path fixtures.
- 92.26% coverage achieved; hard gate is 80%, enforced by a Claude Code PreToolUse hook that blocks git push below that threshold.

```
wildix-MacBook-Air:homework-6 wildix$ pytest --cov=pipeline --cov=frontend --cov=mcp --cov-report=term-missing --cov-fail-under=80
ERROR: usage: pytest [options] [file_or_dir] [file_or_dir] [...]
pytest: error: unrecognized arguments: --cov=pipeline --cov=frontend --cov=mcp --cov-report=term-missing --cov-fail-under=80
initfile: /Users/wildix/WorkingProjects/gen-ai-software-engineering/homework-6/pytest.ini
rootdir: /Users/wildix/WorkingProjects/gen-ai-software-engineering/homework-6

wildix-MacBook-Air:homework-6 wildix$
wildix-MacBook-Air:homework-6 wildix$ python3 -m pytest --cov=pipeline --cov=frontend --cov=mcp --cov-report=term-missing --cov-fail-under=80
===== test session starts =====
platform darwin -- Python 3.11.5, pytest-7.1.1, pluggy-1.6.0
rootdir: /Users/wildix/WorkingProjects/gen-ai-software-engineering/homework-6
configfile: pytest.ini
plugins: cov-7.1.0, anyio-4.14.1, asyncio-1.4.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 38 items

tests/test_fraud_detector.py ..... [ 15%]
tests/test_frontend.py .. [ 23%]
tests/test_mcp_server.py .... [ 34%]
tests/test_models.py ..... [ 47%]
tests/test_orchestrator_integration.py ... [ 57%]
tests/test_results_reader.py ... [ 65%]
tests/test_settlement.py .... [ 76%]
tests/test_validator.py ..... [100%]

===== warnings summary =====
../../../../Library/Python/3.11/lib/python/site-packages/fastapi/testclient.py:1
/Users/wildix/Library/Python/3.11/lib/python/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== tests coverage =====
coverage: platform darwin, python 3.11.5-final-0

Name                               Stmts   Miss  Cover   Missing
-----
frontend/server.py                   17      0  100%
mcp/server.py                        29      1    97%    55
pipeline/_init_.py                   0      0  100%
pipeline/fraud_detector.py           64      3    95%    47, 50-51
pipeline/models.py                   31      0  100%
pipeline/results_reader.py           23      0  100%
pipeline/settlement.py               47      3    94%    39, 42-43
pipeline/validator.py                86     16    81%    13, 107-120, 127

TOTAL                                297     23    92%
Required test coverage of 80% reached. Total coverage: 92.26%
===== 38 passed, 1 warning in 1.32s =====
wildix-MacBook-Air:homework-6 wildix$
```

Real coverage report from this project's test suite.

- The first version of the specification was missing some real-world rules (crash recovery, avoiding duplicate processing, a separate permanent record of every transaction) — found by checking it against common financial-industry standards.
- Interesting discovery, not really a challenge: the spec-writing command didn't know about those rules once added, because they only lived in the specification file itself, not anywhere the command read from. Fixed by moving the rules into the command.
- Two copies of the same tool (context7) were configured at once, causing a conflict warning. Removed the extra one.
- One instruction said "clear the shared folders," but one folder holds a permanent record that must never be deleted. Added a clear note about which folders are safe.

Testing the spec-writing command on purpose and finding the issue.